

Designable by Design: Toward High-Designability Proteins via Reinforcement Learning

Mohammed Ehab, Kasra Mazaheri

Abstract—Protein design is currently one of the most influential applications of diffusion models. However, vanilla diffusion model training on protein datasets tends to output many proteins we can not design in practice. Many SOTA methods use an unprincipled low-temperature scaling trick to improve designability at the expense of sacrificing diversity. In this report, we show that there is no inherent tradeoff between designability and diversity, offer a more principled framework for increasing designability without sacrificing diversity, devise multiple methods to implement that framework, and show some experimental results for our methods.

I. INTRODUCTION

Novel protein design, the practice of designing proteins with novel functions and properties from scratch, is a critical frontier in molecular biology, with many applications in the therapeutic industries [1], [2], [3]. Achieving this goal requires a deep understanding of the complex relationship between protein structure and function, as well as the ability to design novel proteins that can fold into the desired structure. Recent advancements in computational biology, particularly deep learning, have unlocked a new path to systematically generating novel proteins [4], [5], [6], [7], [8]. Among these, diffusion models [9], [10] have emerged as a promising tool for generating protein backbones, where the challenge lies not only in generating novel structures but also in ensuring that the generated structures are functional, stable, and most crucially, **designable**.

In the context of protein design, designability refers to the feasibility with which a protein structure can be folded into a specific structure in 3D space, which determines its function. Moreover, it is also vital for the novelty of the generated proteins that the generative models span a range of structural diversity, while still maintaining high designability, playing a pivotal role in evaluating the success of generative protein models.

Within this context, SOTA algorithms [7], [8] often do a tradeoff based on low-temperature sampling: using a controlling variable γ as a *knob* to guide the model to tradeoff between designability and diversity, by tilting the sampling distribution in a mathematically unprincipled manner.

In this work, we examine the γ trick used in SOTA models through a mathematical lens and shed some light on the distribution it’s attempting to sample from. We, moreover, argue that this tradeoff between designability and diversity is not an inherent part of the generative protein design process, and employ a reinforcement learning (RL)-inspired paradigm to mathematically tilt the sampling distribution in a characterized, and flexible way, improving protein designability all while bypassing the tradeoff for diversity.

Formally, we state our contributions as follows:

- We shed some light on the distribution SOTA methods attempt to sample from.
- We observe there is no inherent tradeoff between designability and diversity. We build upon that by providing a more principled framework to improve the sampling quality based on distribution tilting methods inspired from RL.
- We design our own distribution tilting method that could more broadly be applied to general RL for diffusion problems.

II. UNDERSTANDING THE LOW-TEMPERATURE SAMPLING

In this report, we will focus on Proteina [8], a SOTA flow-based protein design model that offers both unconditional and conditional protein generation. While our work is mathematically model-agnostic (as far as the model is diffusion-based) and applies to both unconditional and conditional sampling, for brevity of the discussion, we investigate our contributions specifically with respect to the Proteina model and unconditional sampling only. Our approaches can be easily extended to work with other models.

Concretely, Proteina is a flow-based model and employs the following reverse process for samplings:

$$dx_t = v_t^\theta(x_t)dt + g(t)s_t^\theta(x_t)dt + \sqrt{2g(t)\gamma}d\mathcal{W}_t. \quad (1)$$

This process looks fairly standard, except for the parameter γ , typically chosen ≤ 1 , that allows the sampling process to add varying (less) amounts of noise than the reverse process suggests. Intuitively, this puts more weight on higher-likelihood trajectories, hoping that designable proteins have higher likelihoods under the model to improve designability. This, however, comes at the expense of sacrificing low-likelihood designable proteins and hence decreasing design diversity.

We will now shed some light on how the γ parameter is performing low-temperature sampling. It is known in the flow matching literature that the following family of generative processes sample from the same distribution [11]

$$dx_t = v_t(x_t)dt + a(t)s_t(x_t)dt + b(t)d\mathcal{W}_t,$$

so long as $a(t) = \frac{1}{2}b(t)^2$. Therefore, the following generative process also samples from the same distribution as the Proteina sampler:

$$dx_t = v_t^\theta(x_t)dt + g(t)\frac{1}{\gamma}s_t^\theta(x_t)dt + \sqrt{2g(t)}d\mathcal{W}_t.$$

Namely, multiplying the noise by $\sqrt{\gamma}$ has the same effect as scaling the score function down by a factor of γ . In Langevin dynamics where there are no varying levels of noise, this result would be equivalent to sampling from a distribution with density $p^{\frac{1}{\gamma}}$, which is a known way to sharpen a distribution. Unfortunately, noising and sharpening don't commute, so scaling down the score function s_t does not quite give us the score function of $p^{\frac{1}{\gamma}}$ after having added noise.

III. THE DESIGNABILITY-DIVERSITY TRADEOFF ILLUSION

In this section, we dig deeper into how the designability and diversity metrics are computed to understand if this distribution sharpening trick is the best choice.

Designability: Proteina generates the 3D protein backbone structure (X), as coordinates of C_α atoms in the 3D space. Another model (ProteinMPNN [12] in our and Proteina's experiments) is used to sample 8 candidate protein sequences that are likely to fold into a structure similar to X . Then, a final protein-folding model (ESMFold [13] in the experiments) is used to fold these sequences. For each sequence, the root-mean-square distance (RMSD) between the atom positions of the folded protein and X is computed, with the minimum of these numbers being called the *RMSD score* or *scRMSD*. A protein is considered designable if its scRMSD is smaller than 2 angstrom, with the overall designability of the model being the probability of it outputting a designable protein.

Diversity: Diversity, on the other hand, can be defined using multiple metrics (e.g., based on pairwise distances or clustering). The crucial observation, however, is that diversity is only computed among designable sampled proteins, with the philosophy being that maximizing *diversity* by sampling nonfunctional undesignable proteins is a futile and fruitless task. As such, we argue that there is no inherent tradeoff between designability and diversity!

IV. FORMULATION

Mathematically, a more principled way to think about the problem at hand is the following: given a diffusion model that samples from a distribution $p_\theta(x)$, we would like a model that samples from a distribution with density proportional to $p_\theta(x)d(x)$ where d is 1 if x is designable and 0 if not.

The first question to ask is: why didn't the authors just train on only designable proteins? Shouldn't that theoretically achieve the desired distribution? The answer is that there isn't enough training data to properly estimate the score function that way. Digging deeper into the training methodology, there are 2 datasets for protein design tasks: Protein Databank (PDB) which contains experimentally-decided structures and AlphaFold DataBank (AFDB) which

contains proteins folded using AlphaFold from a database of sequences. PDB has the issues of being fairly small and biased to proteins that are easy to crystallize, and AFDB has the issue of having relied on an AI model to generate the structures.

SOTA methods such as Proteina and Genie 2 [7] use the AFDB dataset with some data cleaning steps to train their models and then use the low-temperature scaling trick to improve the sampling. With that understanding in mind, these models obtain a better density estimate over designable proteins than a vanilla diffusion model trained on only designable proteins, and it would be better if we can use *their* density estimate and restrict it to designable proteins as formulated above. That formulation lends itself nicely to the recently-proposed distribution tilting methods we will now discuss.

V. BACKGROUND ON DISTRIBUTION TILTING

The problem of fine-tuning diffusion models based on reward functions has received a lot of attention recently. Formally, given a diffusion model with a base sampling distribution p_{base} and a reward function r , the objective is to fine-tune the model to sample from the following distribution instead:

$$p_{tar}(x) \propto p_{base}(x)e^{r(x)}.$$

In [14], they show that p_{tar} is the maximizer of the following objective

$$p_{tar} = \arg \max_{\theta} \mathbb{E}_{x \sim p_\theta} [r(x)] - \text{KL}(p_\theta || p_{base}).$$

Intuitively, the first term maximizes the reward and the second term ensures the distribution remains close to the base one.

To maximize the objective, the authors then take an approach based on stochastic optimal control. They parameterize a control function u_θ and drive the diffusion SDE to sample from p_{tar} . They show that if the SDE is modified to be

$$dx_t = (f(t, x_t) + u_\theta(t, x_t))dt + \sigma_t dw_t,$$

the objective can be rewritten in terms of r and a term quadratic in u , which the optimal control literature had found solutions for. [15] then follows up on their work and finds an even more elegant approach to make the method less involved, still based on SOC.

Notice the shear power of these approaches for post-training diffusion models. Before understanding the lack of designability-density tradeoff, we were interested in more principled ways of sharpening the distribution and noticed that by setting the reward to $\frac{1}{\gamma} \log p_\theta(x) - 1$, we can obtain a more principled way to sample from the density $p_{base}^{\frac{1}{\gamma}}$. However, with our new understanding and the power granted by that tool, we wanted to design reward functions that are functions of scRMSD.

VI. METHODS

We faced a key hurdle trying to apply the SOC fine-tuning methods above to our problem. Namely, the optimal control solver requires $\nabla_x r(x)$. However, the scRMSD metric is not a priori differentiable, since in the middle of its computation, 8 discrete amino-acid sequences are sampled. We tackle this problem using 2 different approaches: in the first approach, we show it is possible to differentiate the scRMSD metric. In the second and more general approach, we devise a novel fine-tuning method to sample from the tilted distribution that does not require reward gradients.

A. scRMSD Differentiation

Let x denote a protein backbone structure and y denote an amino acid sequence. Let $\pi(y|x)$ denote the probability ProteinMPNN samples the sequence y given the backbone structure x . Let $R(x, y)$ denote the RMSD error between x and the result of folding y using ESMFold. Then:-

$$\text{scRMSD}(x) = \mathbb{E}_{y_i \sim \pi(\cdot|x)} \left[\min_{1 \leq i \leq 8} R(x, y_i) \right]$$

To abstract away the multi-sequence sampling, let $\pi^8(\cdot|x)$ denote the product distribution of the 8 sequences and let $R_8(x, y) = \min_{i=1}^8 R(x, y_i)$, where y now is a list of 8 sequences. Then:-

$$\text{scRMSD}(x) = \mathbb{E}_{y \sim \pi^8(\cdot|x)} [R_8(x, y)]$$

$$\begin{aligned} \nabla_x \text{scRMSD}(x) &= \nabla_x \int R_8(x, y) \pi^8(y|x) dy \\ &= \int (\nabla_x R_8(x, y) \pi^8(y|x) + R_8(x, y) \nabla_x \pi^8(y|x)) dy \\ &= \int (\nabla_x R_8(x, y) + R_8(x, y) \nabla_x \log \pi^8(y|x)) \pi^8(y|x) dy \\ &= \mathbb{E}_{y \sim \pi^8(\cdot|x)} [\nabla_x R_8(x, y) + R_8(x, y) \nabla_x \log \pi^8(y|x)]. \end{aligned}$$

ProteinMPNN autoregressively samples amino acids, giving us direct access to $\pi(\cdot|x)$ and the ability to evaluate this expectation.

B. Distribution Tilting without Reward Gradients

In the SOC formulation above, the authors parameterize a control function u to change the continuous time process, giving rise to an optimal control problem that requires the gradients of the reward. We will show that if the KL divergence term is taken over the discretized diffusion processes, it becomes possible to directly write the gradient of the objective w.r.t. θ in a way that does not require the gradient of the reward! Furthermore, the same proof from [14] applies, and we get that if p_{tar} is the minimizer of the objective, then

$$p_{tar}(x) \propto p_{base}(x) e^{r(x)},$$

where p_{base} now is the discretized diffusion process.

To set up the notation, let $x = (x_0, \dots, x_T)$ denote a diffusion trajectory. And let $p_{base}(x)$ denote the likelihood of that **whole trajectory** under the base diffusion process. Similarly, let $p_\theta(x)$ denote the same for the diffusion process p_θ . We get:-

$$\begin{aligned} &\nabla_\theta \mathbb{E}_x [r(x_T) - \text{KL}(p_\theta(x) || p_{base}(x))] \\ &= \nabla_\theta \mathbb{E}_x [r(x_T) - \log p_\theta(x) + \log p_{base}(x)] \\ &= \nabla_\theta \int [r(x_T) - \log p_\theta(x) + \log p_{base}(x)] p_\theta(x) dx \\ &= \int [r(x_T) - \log p_\theta(x) + \log p_{base}(x)] \nabla_\theta p_\theta(x) \\ &\quad - \nabla \log p_\theta(x) p_\theta(x) dx \\ &= \int [r(x_T) - \log p_\theta(x) + \log p_{base}(x)] \nabla_\theta \log p_\theta(x) p_\theta(x) \\ &\quad - \nabla \log p_\theta(x) p_\theta(x) dx \\ &= \int [r(x_T) - \log p_\theta(x) + \log p_{base}(x) - 1] \nabla_\theta \log p_\theta(x) p_\theta(x) dx \\ &= \mathbb{E}_x [[r(x_T) - \log p_\theta(x) + \log p_{base}(x) - 1] \nabla_\theta \log p_\theta(x)] \end{aligned}$$

Now,

$$\nabla_\theta \log p_\theta(x) = \sum_{i=1}^T \nabla_\theta \log p_\theta(x_i | x_{i-1}),$$

and since each one of these transitions is Gaussian, we can evaluate the expectation!

A similar idea was explored in [16], except they only maximize the reward and have no KL divergence term, leading to reward-hacking if over-optimized and no connection to the tilted distribution.

VII. EXPERIMENTS AND FINDINGS

We ran experiments with both methods. The code for the SOC method can be found here and for the KL divergence method can be found here.

A. The SOC Method

In our experiments on the method inspired by the SOC formulation, we adopted the methodology of [15] which use adjoint matching to fine-tune flow. In their work, [15] prove that a very specific *memoryless* noise schedule must be enforced during fine-tuning to account for the dependency between the noise variable and the generated samples. Concretely, given a flow matching model characterized as

$$dX_t = b(X_t, t)dt + \sigma(t)dB_t, X_0 \sim \mathcal{N}(0, I)$$

where

$$b(x, t) = \kappa_t x + \left(\frac{\sigma(t)^2}{2} + \eta_t \right) s(x, t)$$

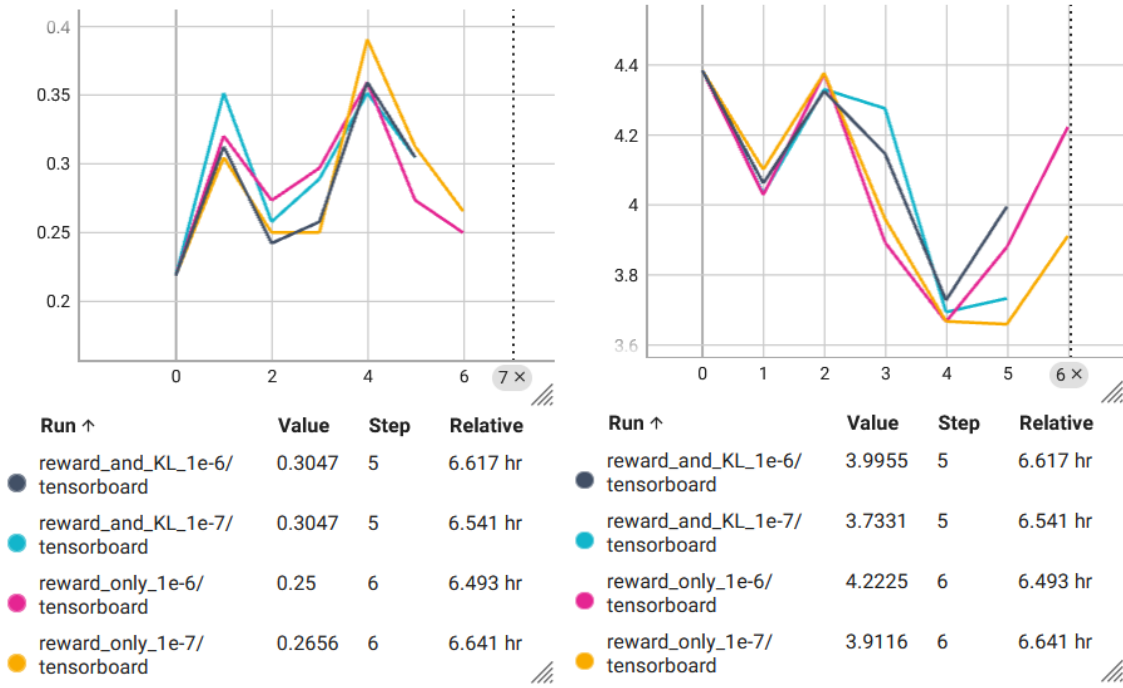


Fig. 1: Designability and average scRMSD for 4 runs, with and without KL divergence and learning rates 10^{-6} and 10^{-7} .

and

$$\kappa_t = \frac{\dot{\alpha}_t}{\alpha_t}, \eta_t = \beta_t \left(\frac{\dot{\alpha}_t}{\alpha_t} \beta_t - \dot{\beta}_t \right),$$

they prove that the memoryless noise schedule is derived as $\sigma(t) = \sqrt{2\eta_t}$.

We use this result and apply it to the flow matching framework of Proteina 1. Coinciding the two mathematical formulations, we derive the memoryless noise schedule for Proteina as

$$g(t) = \eta_t = \frac{1-t}{t}, \gamma = 1.$$

After receiving provisional access to the [15] codebase, we adapted the adjoint matching, and generally the SOC, algorithm to work with diffusion-based protein design, and specifically Proteina, as the original implementation of the paper was heavily tied with Stable Diffusion as its core model. Having overcome these difficulties, we unfortunately ran into additional challenges with this adoption due to Proteina’s (and generally all protein design models’) high number of denoising steps. Specifically, the SOC algorithm from [15] requires tracking denoising trajectories for the adjoint matching process, and backpropagating reward gradients through entire trajectories. However, given that Proteina uses 400 denoising steps in the reverse process, a figure $8\times$ that of Stable Diffusion, this trajectory tracking and backpropagating process becomes computationally challenging due to GPU memory limitations. In our experiments, we attempted to run this process with a batch size of 1, leading to GPU out of memory issues.

Unfortunately, we did not have access to GPUs with larger memory sizes in our experiments and were therefore not able to fully carry out this experiment within the SOC framework of adjoint matching. That said, we have established the theoretical groundwork for this experiment and implemented the necessary algorithms and are only waiting to find access to the necessary computational resources to run the experiments.

B. The Reward-Gradient-Free Method

We ran experiments of both our method and the more vanilla reward maximization without KL divergence method as a control.

We ran into a similar issue as above where the step count was too big to backprop through the whole diffusion process, but we mitigated it by computing the gradient step-by-step and accumulating it. Namely, the gradient above is

$$\mathbb{E}_x \left[\sum_{i=1}^T \nabla_{\theta} \log p_{\theta}(x_i | x_{i-1}) [r(x_T) - \log p_{base}(x) + \log p_{\theta}(x) - 1] \right].$$

We mitigated the issue by first computing $\log p_{\theta}(x)$ and $\log p_{base}(x)$ without gradients and then computing each term of the sum independently and clearing the computation graph after.

We used the reward function

$$r(x) = \sigma(8 - 4 \cdot \text{scRMSD}(x)),$$

which reflects that an scRMSD of 2 angstrom is at the edge of designability and sharply changes around that threshold.

We ran our experiments on a V100 GPU with a batch size of 128 for gradient estimation. Each training epoch took

around an hour to complete. We considered using importance sampling and gradient clipping to speed up the training, similar to [16], but found that due to having to re-evaluate the log likelihood under the new model, employing these methods would have added another tunable hyper-parameter and complexities with minimal gains in performance, even if we were not resampling new samples.

For our first round of experiments, we tried learning rates of 10^{-5} and $4 \cdot 10^{-6}$ for both methods, but we found that the loss increased and designability decreased even with just the objective of maximizing rewards. We concluded the learning rates were too high and are currently running experiments with learning rates 10^{-6} and 10^{-7} that seem a lot more stable and show promising results for the starting epochs that we attach in Figure 1.

C. Discussion of the Challenges of High Number of Denoising Steps

As we already mentioned above, Proteina’s use of 400 steps for sampling has posed implementation challenges, but it’s also posing further challenges with the training. All the RL fine-tuning papers we are citing fine-tune Stable Diffusion [17], which while 4 times bigger than Proteina in parameter count, uses only 20–50 sampling steps. Since the training hinges on sampling and evaluating log-likelihoods, this makes fine-tuning Proteina and performing iteration cycles significantly slower.

Furthermore, since the objective is an expectation over trajectories, its variance scales poorly with the number of sampling steps, requiring even more computational resources to alleviate this problem.

We believe that using a generative protein design model at the production-quality of Stable Diffusion would be far more suitable for this fine-tuning undertaking, at least with the kind of compute we have at hand.

VIII. FUTURE WORK

Since the beginning of this project, we were aware that the problem that we were tackling was a challenging one to solve. We believe that this project will bear more fruition in the future, given the time and resources. Once we secure the necessary higher-memory GPUs for our SOC-based method, we plan to return to this experiment and hope to see similar results extend to protein design models. Furthermore for Reward-Gradient-Free experiment, we are currently looking for ideal learning rates. Once we have converged on that and show a proof of concept for the idea, we hope to acquire more compute to fully fine-tune the model with our approach. Lastly, our proposed distribution tilting method remains an important area for future exploration and we plan to experiment with it on the more familiar and tested Stable Diffusion benchmarks to see its performance relative to the existing methods.

REFERENCES

- [1] “De novo design of picomolar SARS-CoV-2 miniprotein inhibitors | Science.” [Online]. Available: <https://www.science.org/doi/10.1126/science.abd9909>
- [2] D.-A. Silva, S. Yu, U. Y. Ulge, J. B. Spangler, K. M. Jude, C. Labão-Almeida, L. R. Ali, A. Quijano-Rubio, M. Ruterbusch, I. Leung, T. Biary, S. J. Crowley, E. Marcos, C. D. Walkey, B. D. Weitzner, F. Pardo-Avila, J. Castellanos, L. Carter, L. Stewart, S. R. Riddell, M. Pepper, G. J. L. Bernardes, M. Dougan, K. C. Garcia, and D. Baker, “De novo design of potent and selective mimics of IL-2 and IL-15,” *Nature*, vol. 565, no. 7738, pp. 186–191, Jan. 2019, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41586-018-0830-7>
- [3] A. Shانهsazzadeh, M. McPartlon, G. Kasun, A. K. Steiger, J. M. Sutton, E. Yassine, C. McCloskey, R. Haile, R. Shuai, J. Alverio, G. Rakocovic, S. Levine, J. Cejovic, J. M. Gutierrez, A. Morehead, O. Dubrovskiy, C. Chung, B. K. Lutton, N. Diaz, C. Kohnert, R. Consbruck, H. Carter, C. LaCombe, I. Bist, P. Vilaychack, Z. Anderson, L. Xiu, P. Bringas, K. Alarcon, B. Knight, M. Radach, K. Bateman, G. Kopeck-Belliveau, D. Chapman, J. Bennett, A. B. Ventura, G. M. Canales, M. Gowda, K. A. Jackson, R. Caguiat, A. Brown, D. G. d. Silva, Z. Guo, S. Abdulhaqq, L. R. Klug, M. Gander, E. Yapici, J. Meier, and S. Bachas, “Unlocking de novo antibody design with generative artificial intelligence,” Jan. 2024, pages: 2023.01.08.523187 Section: New Results. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2023.01.08.523187v4>
- [4] Y. Qin, M. He, C. Yu, M. Ni, X. Liu, and X. Bo, “The dance of atoms: De novo protein design with diffusion model,” *arXiv preprint arXiv:2504.16479*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.16479>
- [5] K. R. W. J. A. W. S. P. V. P. K. I. L. GR, M.-B. FS, A. I. H. IR, M. R. V. D. L. X. S. GA, H. A. H. CN, K. A. B. E. B. AK, B. M. D. F. and B. D., “De novo protein design with a denoising diffusion network independent of pretrained structure prediction models,” *Nature Biotechnology*, vol. 42, no. 5, pp. 1059–1066, 2024. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/39384986/>
- [6] A. Winnifurth, C. Outeiral, and B. Hie, “Generative artificial intelligence for de novo protein design,” *arXiv preprint arXiv:2310.09685*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.09685>
- [7] Y. Lin, M. Lee, Z. Zhang, and M. AlQuraishi, “Out of Many, One: Designing and Scaffolding Proteins at the Scale of the Structural Universe with Genie 2,” May 2024, arXiv:2405.15489 [q-bio]. [Online]. Available: <http://arxiv.org/abs/2405.15489>
- [8] T. Geffner, K. Didi, Z. Zhang, D. Reidenbach, Z. Cao, J. Yim, M. Geiger, C. Dallago, E. Kucukbenli, A. Vahdat, and K. Kreiss, “Proteina: Scaling Flow-based Protein Structure Generative Models,” Mar. 2025, arXiv:2503.00710 [cs]. [Online]. Available: <http://arxiv.org/abs/2503.00710>
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [10] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” 2015. [Online]. Available: <http://arxiv.org/abs/1503.03585>
- [11] “Diffusion Meets Flow Matching.” [Online]. Available: <https://diffusionflow.github.io/>
- [12] J. Dauparas, I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, A. Courbet, R. J. d. Haas, N. Bethel, P. J. Y. Leung, T. F. Huddy, S. Pellock, D. Tischer, F. Chan, B. Koepnick, H. Nguyen, A. Kang, B. Sankaran, A. K. Bera, N. P. King, and D. Baker, “Robust deep learning based protein sequence design using ProteinMPNN,” Jun. 2022, pages: 2022.06.03.494563 Section: New Results. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2022.06.03.494563v1>
- [13] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, A. d. S. Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives, “Language models of protein sequences at the scale of evolution enable accurate structure prediction,” Jul. 2022, pages: 2022.07.20.500902 Section: New Results. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2022.07.20.500902v1>
- [14] M. Uehara, Y. Zhao, K. Black, E. Hajiramezani, G. Scalia, N. L. Diamant, A. M. Tseng, T. Biancalani, and S. Levine, “Fine-Tuning of Continuous-Time Diffusion Models as Entropy-Regularized Control,” Feb. 2024, arXiv:2402.15194 [cs]. [Online]. Available: <http://arxiv.org/abs/2402.15194>
- [15] C. Domingo-Enrich, M. Drozdal, B. Karrer, and R. T. Q. Chen,

“Adjoint Matching: Fine-tuning Flow and Diffusion Generative Models with Memoryless Stochastic Optimal Control,” Jan. 2025, arXiv:2409.08861 [cs]. [Online]. Available: <http://arxiv.org/abs/2409.08861>

- [16] K. Black, M. Janner, Y. Du, I. Kostikov, and S. Levine, “Training Diffusion Models with Reinforcement Learning,” Jan. 2024, arXiv:2305.13301 [cs]. [Online]. Available: <http://arxiv.org/abs/2305.13301>
- [17] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” Apr. 2022, arXiv:2112.10752 [cs]. [Online]. Available: <http://arxiv.org/abs/2112.10752>